KINNAIRD COLLEGE FOR WOMEN

UNIVERSITY ,LAHORE

SUBJECT: *DIGITAL LOGIC DESIGN*

ASSIGNMENT: *PRE-MID*

SUBMITTED ON*: 2-02-2025*

SUBMITTED BY*: AIMAN TAHIR*

SHAFAQ REHMAN

SUBMITTED TO*: MAM ARFA DILAWARI*

TOPIC : *The universal gates and implementation of basic gates* ***NAND AND NOR*** *gates*

SESSION*: FALL 2024*

SECTION: B

1: INTRODUCTION TO A UNIVERSAL GATES

QUESTION 1:

Provide an overview of universal gates in digital logic design

ANSWER:

Universal Gates in Digital Logic

Universal gates are logic gates that can be used to implement any Boolean function. The two universal gates are:

NAND Gate:

Boolean Expression: Y=(A·B) ' Can be used to create NOT, AND, OR, XOR gates. Most commonly used in circuit design due to its simplicity and efficiency.

NOR Gate

Boolean Expression: Y=(A+B) '

Can be used to create NOT, OR, AND, XNOR gates. Preferred in some memory and processor designs.

These gates are called universal because they can be used to design any digital circuit. They are widely used in processors, memory units, and logic circuits.

QUESTION 2:

Explain the concept of universal gates and their significance in simplifying circuit design.

ANSWER:

Universal Gates and Their Significance in Circuit Design Universal gates are logic gates that can be used to implement any Boolean function or digital circuit without needing additional gate types. The two universal gates are:

NAND Gate Y=(A·B) ' – Performs an AND operation followed by NOT

NOR Gate Y=(A+B) ' – Performs an OR operation followed by NOT

Why Are They Called Universal?

Any logic function (AND, OR, NOT, XOR, XNOR) can be built using only NAND or NOR gates . They provide a simpler and more efficient way to design circuits .Significance in Circuit Design

Simplification of Circuits: *Instead of using multiple gate types, circuits can be designed using only NAND or NOR, reducing complexity.*

Cost-Effective Manufacturing: *Using a single type of gate reduces production costs in hardware design.*

Reliability & Efficiency: *NAND gates are widely used in microprocessors, memory units, and digital circuits due to their speed and low power consumption.*

Error Reduction: *Standardizing circuits with universal gates minimizes design errors and improves consistency.*

Because of these advantages, universal gates are a fundamental building block in digital electronics and computing.

QUESTION 3:

Explain why NAND and NOR gates are considered universal gates•

ANSWER:

NAND and NOR gates are called universal gates because they can implement any Boolean function using only one type of gate.

Reasons for Universality

Can Create Basic Gates:

NOT using NAND: A′=NAND (A, A) A′=NAND(A , A)

NOT using NOR: A′=NOR (A , A)A′=NOR(A,A)

Combining them forms AND, OR, XOR, and XNOR gates.

Simplifies Circuit Design:

Using only one type of gate reduces complexity and cost.

Practical Use:

NAND is widely used in processors and memory;

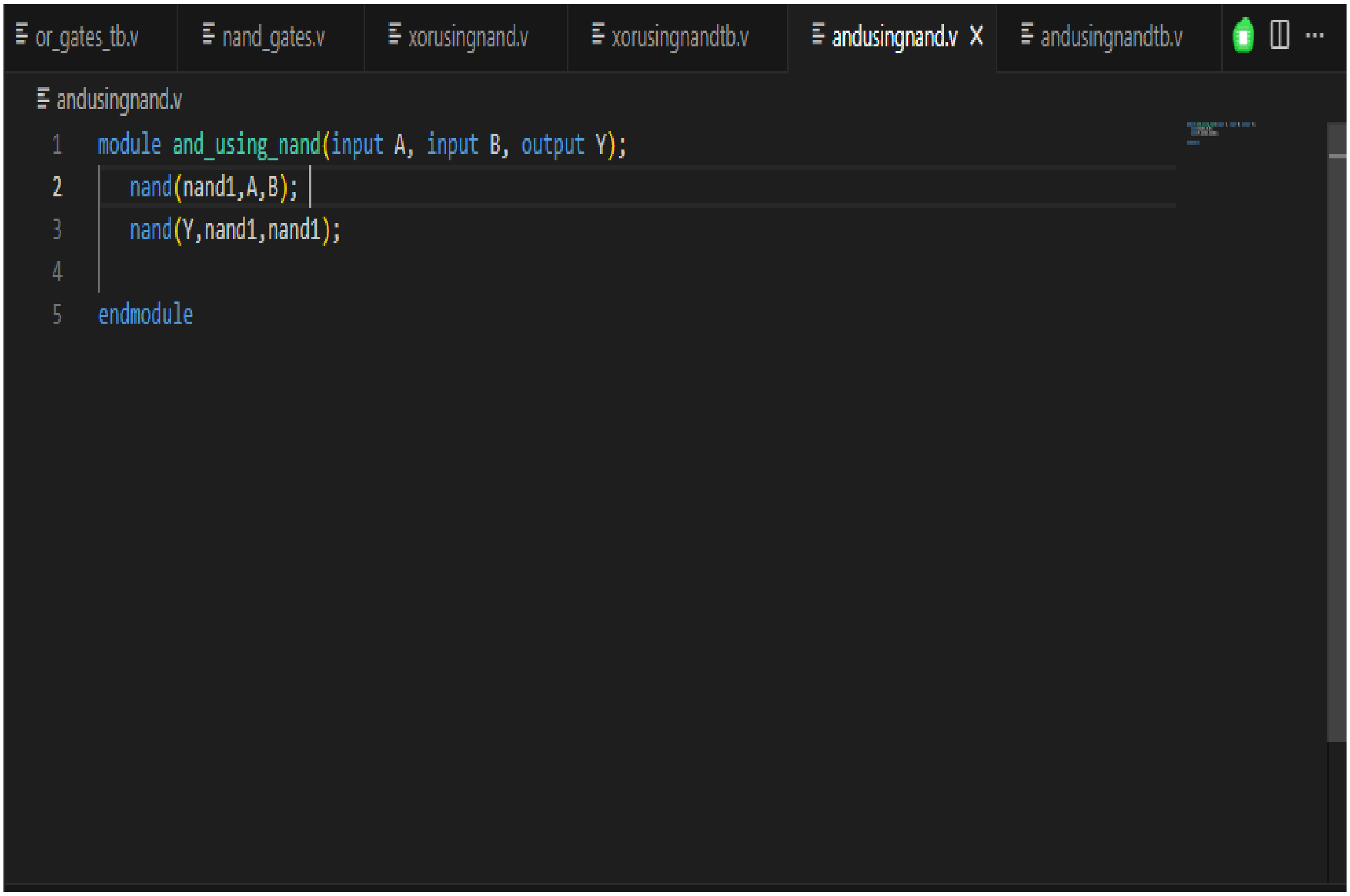
NOR is essential for flip-flops and control circuits. Since they can replicate all logic functions, NAND and NOR are fundamental in digital electronics.

2: Implementation of Basic Gates using NAND Gate in Verilog

QUESTION 1:

design and implement an AND gate using only NAND gates

- ***AND gate using only NAND gate .V:***



or_gates_tb.v
nand_gates.v
xorusingnand.v
xorusingnandtb.v
andusingnand.v
andusingnandtb.v
andusingnand.v
module and_using_nand(input A, input B, output Y);
nand(nand1,A,B);
nand(Y,nand1,nand1);
endmodule

.

- ***TEST BENCH:***

```verilog
`include "andusingnand.v"
module and_using_nand_tb;
    reg a,b;
    wire y;
    and_using_nand test(a,b,y);
    initial begin
        $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
        a=0;b=0;#10;
        a=0;b=1;#10;
        a=1;b=0;#10
        a=1;b=1;#10
        $finish;
    end

endmodule
```

- ***OUPUT:***

```
[Running] andusingnandtb.v
Time=0,a=0,b=0,y=0
Time=10,a=0,b=1,y=0
Time=20,a=1,b=0,y=0
Time=30,a=1,b=1,y=1
andusingnandtb.v:12: $finish called at 40 (1s)
[Done] exit with code=0 in 0.278 seconds
```

- ***WAVEFORM:***

QUESTION 2:

design and implement an XOR gate using only NAND gate

- ***XOR gate using only NAND gate .V:***

xorusingnand.v

```verilog
module xor_using_nand(input A, input B, output Y);
    nand(nand1,A,B);
    nand(nand2,A,nand1);
    nand(nand3,B,nand1);
    nand(Y,nand2,nand3);

endmodule
```

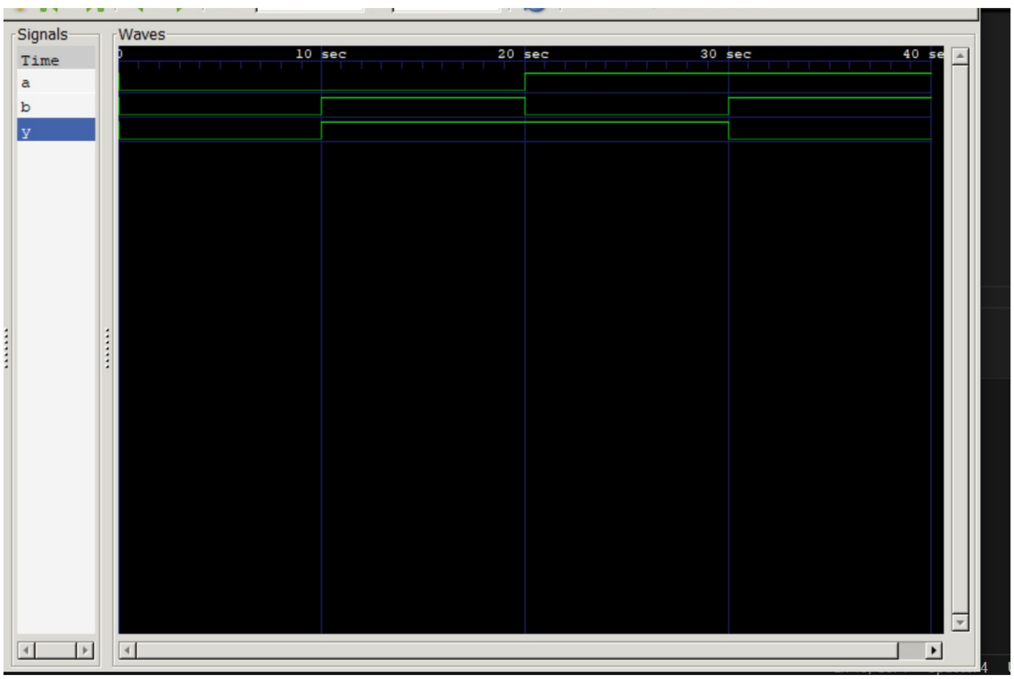
- ***TEST BENCH:***

xorusingnandtb.v

```verilog
`include "xorusingnand.v"
module xor_using_nand_tb;
   reg a,b;
   wire y;
   xor_using_nand test(a,b,y);
   initial begin
        $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
        a=0;b=0;#10;
        a=0;b=1;#10;
        a=1;b=0;#10
        a=1;b=1;#10
        $finish;
   end
   initial begin
        $dumpfile("dump1.vcd");
        $dumpvars();
   end
endmodule
```

OUTPUT:

- ***WAVEFORM:***



QUESTION 3:

design and implement a XNOR gate using only NAND gates.

- ***XNOR gate using NAND gate .V:***

xnorusingnand.v

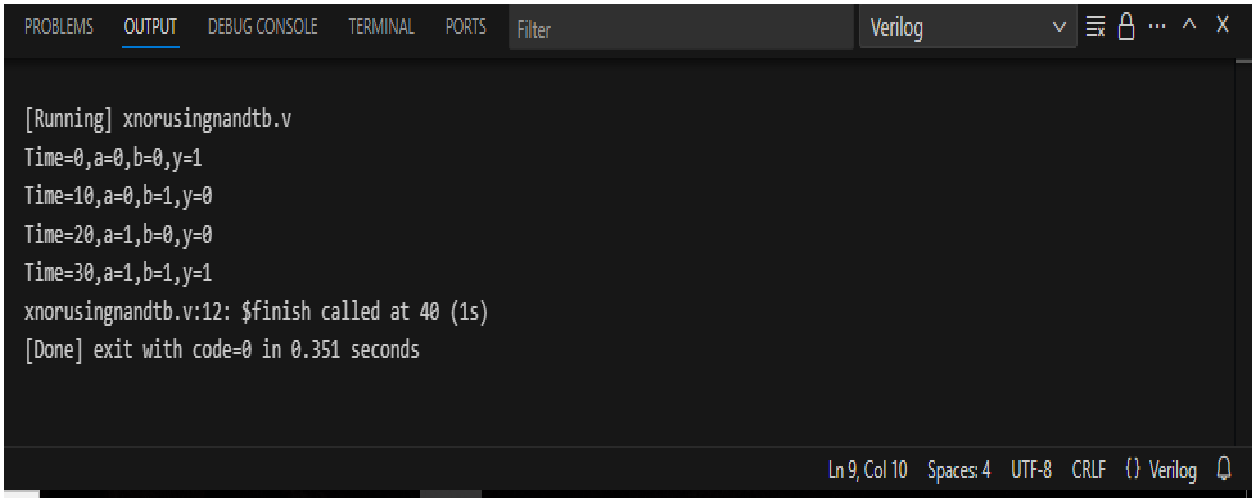
```verilog
module xnor_using_nand(input A, input B, output Y);
  nand(nand1,A,B);
  nand(nand2,A,nand1);
  nand(nand3,B,nand1);
  nand(nand4,nand2,nand3);
  nand(Y, nand4,nand4);


endmodule
```

- ***TEST BENCH:***

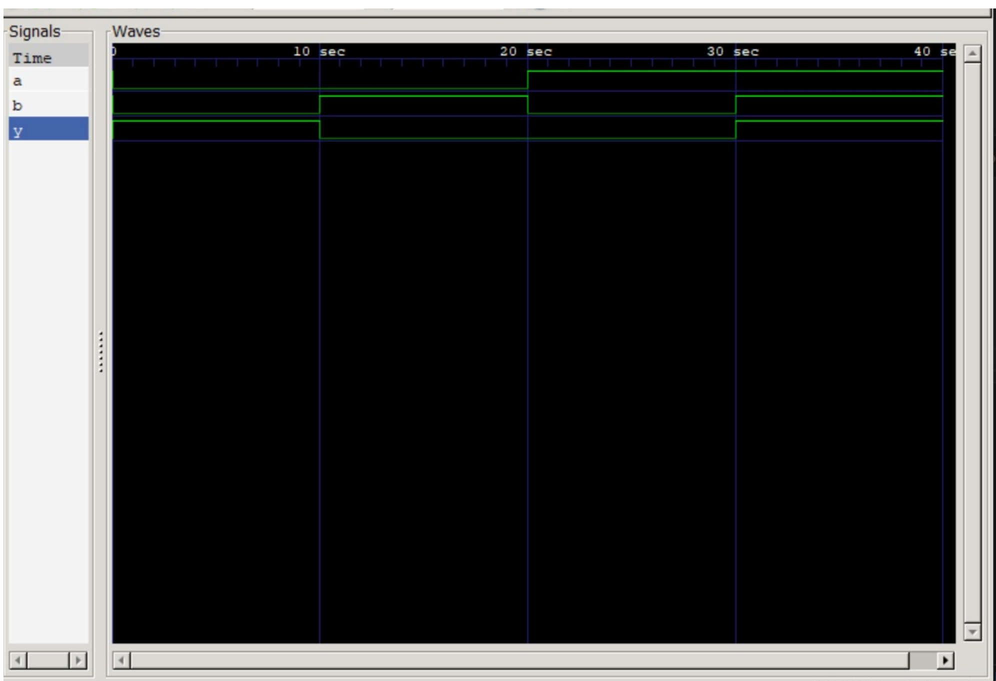
xnorusingnandtb.v

```verilog
`include "xnorusingnand.v"
module xnor_using_nand_tb;
   reg a,b;
   wire y;
   xnor_using_nand test(a,b,y);
   initial begin
        $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
        a=0;b=0;#10;
        a=0;b=1;#10;
        a=1;b=0;#10
        a=1;b=1;#10
        $finish;
   end

endmodule
```

- ***OUTPUT:***



- ***WAVEFORM:***



3: Implementation of Basic Gates using NOR Gate in Verilog

QUESTION 1:

design and implement an AND gate using only NOR gate

- ***AND gate using only NOR gate .V:***

andusingnor.v

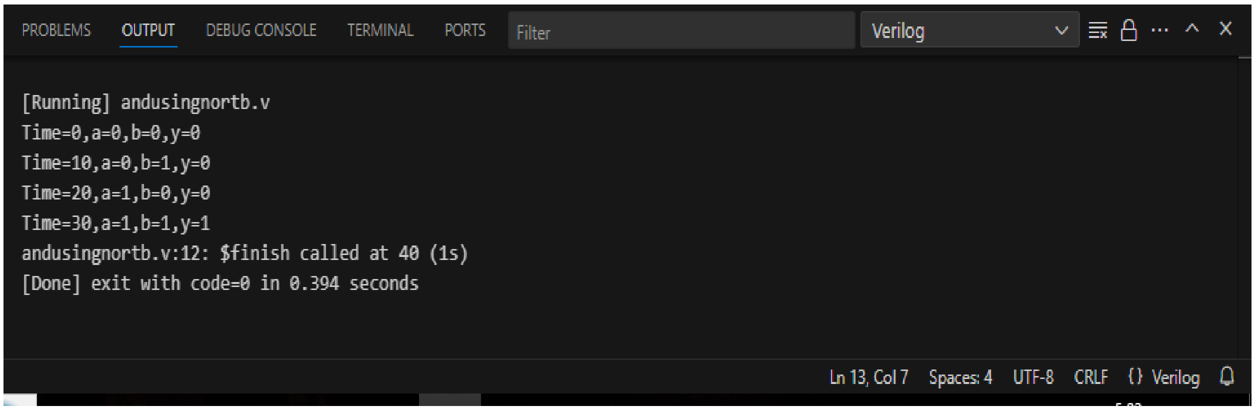
```
module and_using_nor(input A, input B, output Y);
  nor(nor1,A,A);
  nor(nor2,B,B);

  nor(Y,nor1,nor2);


endmodule
```

- ***TEST BENCH:***

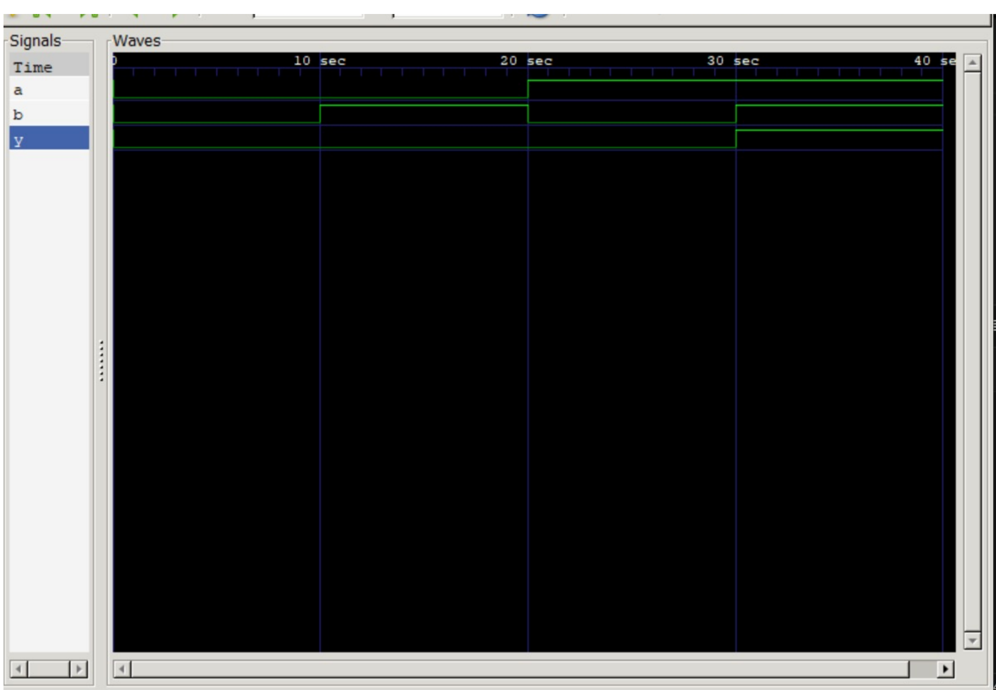
andusingnortb.v

```
`include "andusingnor.v"
module and_using_nor_tb;
  reg a,b;
  wire y;
  and_using_nor test(a,b,y);
  initial begin
       $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
       a=0;b=0;#10;
       a=0;b=1;#10;
       a=1;b=0;#10
       a=1;b=1;#10
       $finish;
  end

endmodule
```

- ***OUTPUT:***



- ***WAVEFORM:***



QUESTION 2:

design and implement an XOR gate using only NOR gates

- ***XOR gate using only NOR gate.V:***

```
module xor_using_nor(input A, input B, output Y);
    nor(nor1,A,B);
    nor(nor2,A,nor1);
    nor(nor3,B,nor1);
    nor(nor4,nor2,nor3);
    nor(Y,nor4,nor4);
endmodule
```

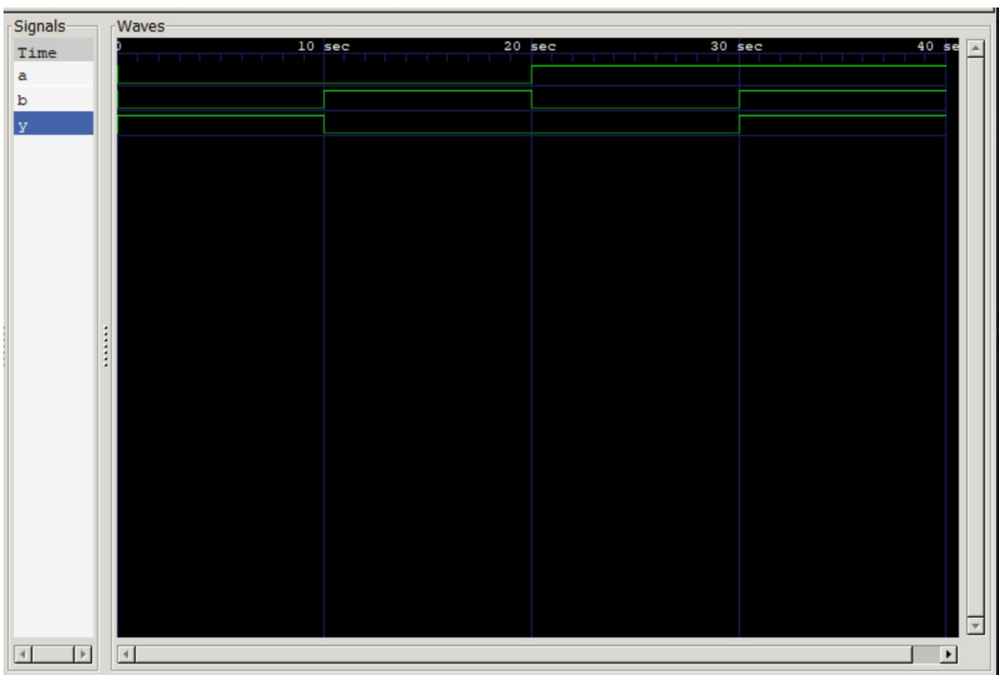
- ***TEST BENCH:***

```
`include "xorusingnor.v"
module xor_using_nor_tb;
   reg a,b;
   wire y;
   xor_using_nor test(a,b,y);
   initial begin
        $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
        a=0;b=0;#10;
        a=0;b=1;#10;
        a=1;b=0;#10
        a=1;b=1;#10
        $finish;
   end

endmodule
```

- ***OUTPUT***

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   Filter   Verilog

[Running] xorusingnortb.v
Time=0,a=0,b=0,y=0
Time=10,a=0,b=1,y=1
Time=20,a=1,b=0,y=1
Time=30,a=1,b=1,y=0
xorusingnortb.v:12: $finish called at 40 (1s)
[Done] exit with code=0 in 0.36 seconds
```

- ***WAVEFORM:***



QUESTION 3:

design and implement a XNOR gate using only NOR gate.

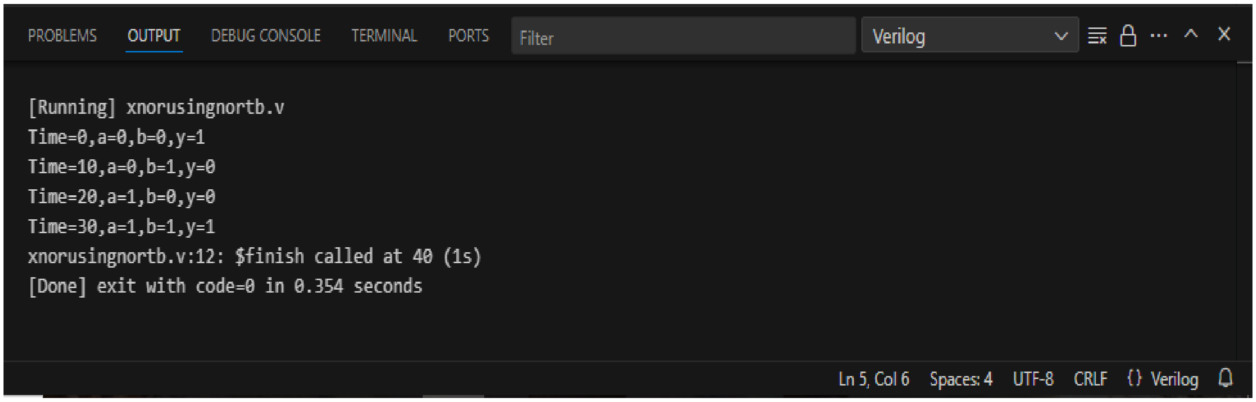
- ## *XNOR gate using only NOR gate.V:*

```
module xnor_using_nor(input A, input B, output Y);
  nor(nor1,A,B);
  nor(nor2,A,nor1);
  nor(nor3,B,nor1);
  nor(Y,nor2,nor3);
endmodule
```

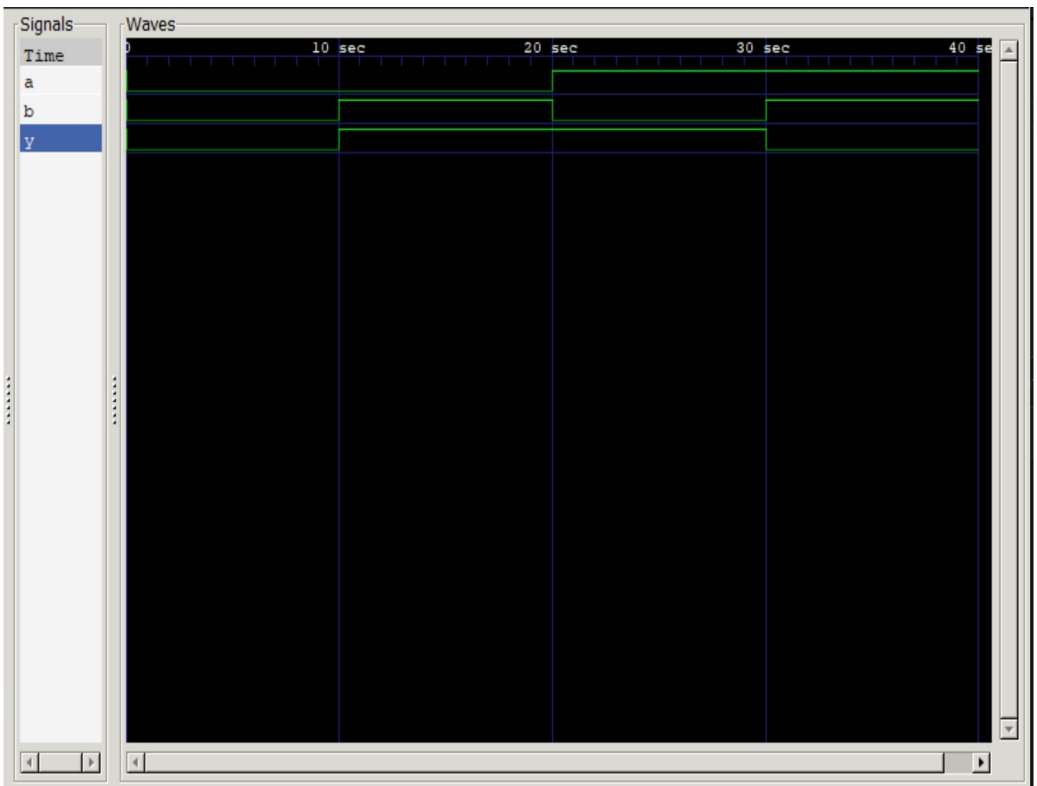
- ## *TEST BENCH:*

```
`include "xnorusingnor.v"
module xnor_using_nor_tb;
  reg a,b;
  wire y;
  xnor_using_nor test(a,b,y);
  initial begin
      $monitor("Time=%0t,a=%b,b=%b,y=%b",$time,a,b,y);
      a=0;b=0;#10;
      a=0;b=1;#10;
      a=1;b=0;#10
      a=1;b=1;#10
      $finish;
  end

endmodule
```

- ## *OUTPUT:*


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Filter
Verilog
[Running] xnorusingnortb.v
Time=0,a=0,b=0,y=1
Time=10,a=0,b=1,y=0
Time=20,a=1,b=0,y=0
Time=30,a=1,b=1,y=1
xnorusingnortb.v:12: $finish called at 40 (1s)
[Done] exit with code=0 in 0.354 seconds
Ln 5, Col 6 Spaces: 4 UTF-8 CRLF {} Verilog

- ***WAVEFORM:***


Signals
Waves
Time
a
b
y
10 sec
20 sec
30 sec